

A Tactic-Based Prover for Dependent Session Types: Exploiting Curry-Howard to Develop Correct-by-Construction Concurrent Processes

Kegan McIlwaine
kmcilwai@uwyo.edu
University of Wyoming
Laramie, WY, USA

James Caldwell
jlc@uwyo.edu
University of Wyoming
Laramie, WY, USA

Abstract

STILL is an implementation of an interactive, tactic-based theorem proving system for dependent session types with polymorphism and coinduction. Our prover extracts π -calculus terms from proofs. STILL provides tactic engines for developing proofs in two dependent type systems: *i.*) the session type layer is used to specify protocols, whose well-formed types are inhabited by processes in an enriched value-passing, polymorphic π -calculus; and *ii.*) the functional layer implements the Extended Calculus of Constructions whose role is to describe computations and data within processes. Session types enjoy a Curry-Howard correspondence with propositions in intuitionistic linear logic. Under the correspondence, propositions are session types and proofs are processes. A significant challenge to developing goal-directed proofs in linear logic involves managing the linear resources. Proof rules for the *multiplicative* operators ($\otimes$, $\multimap$) require an explicit partitioning of the linear context, demanding the resource split before the resource requirements of the two resulting subproofs can be known. STILL implements a novel nondeterministic representation of linear contexts that allows users to delay the partitioning choice. The nondeterminism inherent in partial derivations is resolved in the process of constructing tactic validations for complete derivations. The resulting proof is validated by a small kernel. Notably, our technique requires no modification to the proof rules of the underlying logical system, and can be used to implement other tactic-based provers for substructural logics. We have verified in Isabelle/HOL that our nondeterministic representation of linear contexts is *complete* in the sense that if there is a proof that a session type is inhabited, then there is a tactic proof in STILL verifying the same type is inhabited. We also verified that the system is *sound* — *i.e.* if there is a tactic derivation, consuming all linear resources, of a session type, then there exists a process that inhabits that session type. We have used STILL to extract a number of correct-by-construction processes from proofs that their specifications (session types) are inhabited. To evaluate the usability of STILL, we have provided Claude Opus 4.7 with PDF files of

papers containing the descriptions of three distinct protocols, and the AI was able to formulate a STILL specification of a TCP-like handshake protocol. We instructed the LLM to both specify (in STILL) *and* verify those protocols using STILL's tactic system. It was able to do so in every case, notably without prior training on STILL proofs.

Keywords: interactive theorem proving, session types, linear logic, pi-calculus, dependent types, Curry-Howard

1 Introduction

Since Girard's seminal publication [21], linear logic has provided a logical foundation for diverse computational systems that include constraints on resources. More than two decades later, Caires and Pfenning [8] identified a Curry-Howard correspondence between session types and intuitionistic linear logic. This was later expanded upon by Wadler [51], who established a Curry-Howard correspondence between session types and classical linear logic.

Initially introduced by Honda [28], session types provide a type discipline for Milner's π -calculus [35]. Session types allow for specifications of communication protocols. Toninho et al. [47] have developed a theory of *dependent* session types, based on intuitionistic linear logic, which extends a version of the term-passing π -calculus with polymorphism (type variables) and kinds (a type of types). While work on developing dependent session types continues [49], the work on applying these powerful theories has not gained as much traction in practice as one might have expected.

A motivation for the work described in this paper is our belief that specifications are easier to write, and more comprehensible, when they are written in richly expressive logics of the kind just mentioned. Implementations of other powerful logical systems [15, 22, 31, 37–41, 46] have been extraordinarily successful. We are unaware of similar systems for the theory of dependent session types.

1.1 Our Contribution

This paper describes STILL¹, a tactic-based theorem prover for dependent session types with coinduction and session

Conference'17, Washington, DC, USA

2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnnn.nnnnnnn>

¹Repository URL withheld for double-blind review. The prover sources are provided as supplementary material.

type polymorphism. STILL stands for Session Types and Intuitionistic Linear Logic. The underlying type system of STILL is based on the dependent session type system presented in [47], with extensions that add polymorphic session types [7] and coinductive session types [48]. Following [47], the functional layer is parametric in the underlying type theory. STILL's architecture reflects the modularity of [47]; we instantiate it with the Extended Calculus of Constructions (ECC) [30], but the architecture is intended to admit other dependently typed systems. We return to this in Section 7.

The work presented in this paper differs from previous work in the following ways:

- To our knowledge, STILL is the first standalone tactic-based interactive theorem prover for a dependent session type system. Previous work on dependent session types either embeds the type discipline in another proof assistant (e.g., the dependent π -calculus embedded in Agda presented by Ciccone and Padovani [14]) or implements type checking for restricted forms of dependency in programming languages (e.g., RAST [17] and Granule [32]).
- The STILL tactic system has been verified using Isabelle/HOL [38] and the existing deep embedding of intuitionistic linear logic in Isabelle/HOL [45]. The verification shows that every derivable formula in the propositional core of intuitionistic linear logic of STILL can be proved by some sequence of STILL tactic applications (completeness). The verification also shows that a sequence of tactics that closes every open subgoal and reserves all resources proves the original goal (soundness). Theorems proved as part of the verification demonstrate that the delayed context splitting at the tactic level is not a simple change in notation. Disjoint multiplicative partitions in complete subgoal trees and the fresh variable requirement of interactive construction are proved as theorems of the discipline.
- Unlike prior systems that achieve delayed partitioning by modifying the inference rules of the underlying calculus [11, 27, 29, 50], STILL preserves the original rules of the system [7, 47, 48] and pushes the nondeterminism into the tactic layer. This is achieved with a nondeterministic representation of context partitions. A traversal of the tree of subgoals is used to hide resources that have been reserved in other branches. Validations reify these trees into proofs. We count this as a significant advantage because of the considerable effort that goes into proving soundness, progress, subject reduction, consistency, normalization, and cut elimination when modifications are made.
- The STILL prover includes a separate tactic engine and kernel for developing proofs and type checking terms in the Extended Calculus of Constructions [30] that integrates into the overall system. Other interactive

systems with two layers of logic include the Abella interactive theorem prover [20], and the Hybrid tool [18] within Isabelle/HOL and Rocq. These systems have a logical layer for specification and a logical layer for reasoning about specifications. STILL differs from the other two-layer systems by providing two object-level dependent type systems inside a standalone theorem prover. The session type layer provides the ability to specify properties of protocols, and the dependently typed functional layer provides the ability to specify properties of the values communicated by those protocols.

- Extracted proof terms are π -calculus terms rather than λ -calculus terms. Prior works that support program extraction from linear type systems either provide a type checker for the π -calculus [14, 53] or are designed to type check and extract terms of a linear λ -calculus [3, 32, 33, 50].

Developing STILL as an embedding in an existing proof assistant (as Ciccone and Padovani [14] do with Agda) would have the advantage of reusing a mature kernel and tactic infrastructure. However, the multiplicative context partitioning problem can only be cleanly addressed when the prover controls how partially committed contexts are presented to the user and verified by the kernel. In the Isabelle embedding of linear logic by Kalvala and De Paiva [29], the inference rule modifications are motivated by the need for additional proof obligations that validate linear context partitioning after the proof tree is fully built. STILL instead represents linear contexts nondeterministically in the tactic layer which has no analogue in the proof state of a host prover. Nondeterministic subgoals can be displayed to the user with available resources visible while differentiating committed and uncommitted resources. Furthermore, the STILL kernel that validates a proof (including partition disjointness) does not depend on the nondeterminism in the tactic layer. A full proof object must use all resources in the linear context.

This paper is organized as follows. Section 2 provides a brief overview of the underlying theory of dependent session types with polymorphism and coinduction that is implemented by STILL. Section 3 describes the architecture of the system, which includes the implementation of tactics with delayed context partitioning and the trusted computing base. Section 4 presents an Isabelle/HOL mechanization of the correctness of the delayed partitioning technique. Section 5 evaluates the usability of the system and demonstrates the application of the prover with example specifications, example extractions of π -calculus terms, and benchmarks for several types of proofs. Section 6 provides an overview of work related to the STILL prover. Section 7 concludes with discussion about the future applications of the STILL prover and the techniques that can be applied to future systems.

2 Dependent Session Types

STILL implements the system of dependent session types of Toninho et al. [47], extended with the coinductive session types of Toninho et al. [48] and the polymorphic session types of Caires et al. [7]. We give an overview of the system here; the full proof rules can be found in their respective works.

A sequent in this system has the form $\Omega; \Psi; \Gamma; \Delta \vdash_{\eta} P :: z : C$. To the left of $\vdash$ are four (finite) contexts mapping variables to types. The contexts track typing judgments of variables and their usage restrictions. The empty context is denoted by the symbol “.”. Ω binds session type variables (used in well-formedness checks for polymorphic session types). Ψ is the *functional context*: it binds variables used in the functional term language, and is used to type values that are sent or received over channels. Γ is the *unrestricted context*: its variables denote channels with replicating (server-like) behavior. Δ is the *linear context*: each of its variables denotes a channel that must be used exactly once. The binding map η records typing contexts for corecursive process variables. Toninho et al. [48] rule out non-productive corecursive processes such as $\text{corec } X().X()$ using syntactic constraints on session types that are enforced by the STILL kernel (see Section 3.3). To the right of $\vdash$, P is a term in an enriched value-passing π -calculus (with value passing, session-type passing, and corecursion), z is the channel implementing the session, and C is its session type. Variable names in Γ , Δ , and $\{z\}$ are pairwise disjoint. We write $\bar{y}$ for a list of variables. For readability we may omit P from the sequent, writing $\Omega; \Psi; \Gamma; \Delta \vdash_{\eta} z : C$.

An example of a protocol that can be expressed and validated in this system is a seller channel z with the type $z : \forall x : \text{int}. \$\text{usd } x \multimap (\text{item} + \$\text{usd } x)$. This describes a seller that receives an amount x , a payment of x dollars, and then either ships the item or refunds exactly x dollars. The refund type depends on the amount received earlier in the protocol. A coinductive auction example that uses these same ideas with a polymorphic item type can be found in Section 5.4.

Figure 1 gives the concrete syntax used by STILL for functional terms, session types, and process terms. The soundness and type preservation for these languages are established in [7, 8, 47, 48] and the functional language is the Extended Calculus of Constructions (ECC) [30]. The π -calculus terms in Section 5 use this syntax.

3 Tactics for Session Types

Tactics were initially described by Gordon et al. [22] as pure functions with the following types:

```
tactic = goal -> ([goal], validation)
validation = [theorem] -> theorem
```

As Gordon et al. state, primitive tactics can be written for logical systems by inverting the inference rules, *i.e.*, reading them bottom-up. A *valid tactic* [22] has the property that if

```
n      ::= (natural numbers)
x,y,z   ::= (variable names)
xs,ys,zs ::= x,xs |  $\epsilon$ 

-- Functional terms
M,N,S,T ::= Pi x : M . N | Sigma x : M . N
          | lambda x : M . N | (M,N) : Sigma x : S . T
          | fst M | snd M | M N
          | Prop | Type n | x

-- Session types
A,B      ::= 1 | A -o B | A * B | A & B | A + B
          | nu x . A | !A | x
          | $M | forall x : M . A | exists x : M . A
          | forall x : stype . A | exists x : stype . A

-- Process terms
P,Q      ::= P|Q | (nu x:A) P
          | x[y].P | x[M].P | x[A].P
          | x.inl;P | x.inr;P | x.case(P, Q)
          | x(y).P | !x(y).P
          | [x <-> y] | [x <- M]
          | corec x(ys) P (zs) | x(ys)
```

Figure 1. Concrete syntax for terms and types in the STILL prover.

the list of theorems given to the validation function proves the list of goals returned as the result of the tactic, then the theorem output from the validation function will prove the goal originally given to the tactic. Now consider the rule for the tensor operator in the system described by Toninho et al. [47]:

$$\frac{\Omega; \Psi; \Gamma; \Delta_1 \vdash_{\eta} y : A \quad \Omega; \Psi; \Gamma; \Delta_2 \vdash_{\eta} z : B}{\Omega; \Psi; \Gamma; \Delta_1, \Delta_2 \vdash_{\eta} z : A \otimes B} \otimes R$$

Applying the inference rule requires two proofs with disjoint linear assumptions Δ_1 and Δ_2 . A naive approach to writing a valid tactic for the rule requires the user to provide a partition of the assumptions when the tactic is applied. The tactic would then output the two subgoals with those user-provided partitions as the respective assumptions in each subgoal. However, this requires the user to have knowledge of exactly how the proof will partition the assumptions. Such a system would require the user to have a clear plan for resource usage in at least one branch. Otherwise, the user may pick the wrong partition, making it impossible to complete the proofs of the subgoals.

3.1 Delaying Partition Choices

In order to delay the partitioning decision, other automated and interactive systems often modify the rules of the logic. Modifications include adding constraints that must be solved to give the desired partition [50], having the sequent indicate the used and unused assumptions in the context that can be used in other branches [1, 11, 26, 53], or creating

backchaining rules that delete resources in the linear context [27]. These modifications still result in proof rules that contain unknown partition results at the top of the inference rules. Constraint solving is deferred, and the leftover resource information is not known until the full proof branch is complete.

To demonstrate the issue, consider a modification to the tensor right rule based on the resource management calculus presented by Cervesato et al. [11]:

$$\frac{\Omega; \Psi; \Gamma; \Delta^I \setminus \Delta' \vdash_{\eta} y : A \quad \Omega; \Psi; \Gamma; \Delta' \setminus \Delta^O \vdash_{\eta} z : B}{\Omega; \Psi; \Gamma; \Delta^I \setminus \Delta^O \vdash_{\eta} z : A \otimes B} \otimes R$$

In this system, the context $\Delta^I \setminus \Delta^O$ indicates that the linear resources in Δ^I are available to prove the goal, and the resources that were not used in the proof are in Δ^O . Although the notation defers commitment to a partition, a bottom-up tactic still requires the user to supply Δ' and Δ^O at the point of application. The choice is not actually delayed.

Conceptually², STILL tactics have the following type:

```
ndGoal = goal set
tactic = ndGoal -> ([ndGoal], validation)
validation = [theorem] -> theorem
```

STILL tactics take as input a (possibly) nondeterministic goal and output a pair containing a list of nondeterministic goals and a validation. In a nondeterministic subgoal, the linear context is stored as all currently unclaimed resources rather than enumerating all possible partitions. For example, the TensorR tactic for the tensor right rule outputs the subgoals for the left and right operands with the full linear context available on both sides. See Section 4.5 for the formalization of the STILL tactic layer with nondeterministic goals.

As tactics are applied, invalid instances of subgoals are pruned from the nondeterministic subgoal. If the collection of nondeterministic subgoals is empty, that shows that the derivation tree is complete — or — there is no well-formed derivation tree resulting from the tactics as applied. Which of these two holds can tentatively be identified if the last rule applied was an axiom rule. But whether the derivation is a valid proof can only be determined by running the validation.

Figure 2 demonstrates an example of proof development beginning with the single subgoal $\vdash_{\eta} a : A, b : B \vdash_{\eta} z : B \otimes A$. Arrows between each step of development indicate application of the tactic the arrow is labeled with. For readability of the figure we use the color red to represent channels that are nondeterministic in the linear context. For example, $\vdash_{\eta} a : A, b : B \vdash_{\eta} z : A$ represents a nondeterministic subgoal with channels a and b shared with another subgoal. Reserving one of these channels will make it unavailable to the other subgoal that currently shares the channel across the multiplicative branch in the subgoal tree.

²In the implementation of STILL, the state monad is used to represent proof states. Thus a tactic is an action on the state monad.

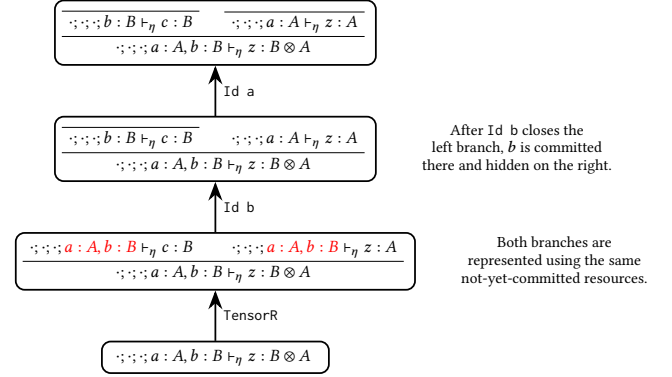


Figure 2. Delayed partitioning during proof construction. The proof is constructed by applying tactics to subgoals at the leaves of a partial derivation.

```
hide(c):
  R := empty set
  p := parent(c)
  if p exists:
    if p was expanded by a multiplicative tactic:
      for each sibling s of c under p:
        R := R union reserved(s)
    R := R union hide(p)
  return R
```

Figure 3. The STILL resource hiding algorithm.

Applying the $\text{Id } b$ tactic reserves b and eliminates b from the other subgoal. Applying the $\text{Id } a$ tactic completes the proof. The STILL prover checks the full derivation before accepting the theorem. Following this simple proof demonstrates how the partition of the linear context is determined as the proof is constructed. The choices are not baked into the rules, and a partitioning choice is not explicitly made by the user.

3.2 Applying Tactics

The STILL proof state contains a subgoal tree from the initial goal to the current open subgoal. Children are added to the tree when a subgoal is expanded by applying a tactic. Each expanded subgoal is also labeled with linear resources that were consumed by the expansion and the branch type of the expansion that was performed. The branch type distinguishes multiplicative and additive expansions. Multiplicative expansions require disjoint linear contexts across the children, and additive expansions require the same linear context.

When the prover needs to display the sequents corresponding to subgoals, it traverses the subgoal tree and collects all resource names that are not available to those subgoals. Those resources are then hidden and treated as unavailable. The

algorithm performing the recursive traversal that collects the hidden resources can be seen in Figure 3. A resource returned by the algorithm has been reserved in a sibling branch of some multiplicative expansion on the path from the root to the current subgoal. Preventing the use of such resources prevents the construction of proofs with invalid partitions. This claim is made precise and proved in Section 4 (Lemma 4.2).

3.3 The Trusted Computing Base

The STILL theorem prover is organized into separate modules for the type system kernels and tactic implementations. The architecture diagram of the STILL prover can be seen in Figure 4 where nodes represent system components (typically a single Haskell module). Arrows between nodes indicate the source node references the target node. Nodes colored in blue represent the trusted computing base of the system. These are the two kernels for proof checking: one for the system of dependent session types described in Section 2, and one for the Extended Calculus of Constructions (ECC) [30]. These kernels check the correctness of proof objects in the two systems. The dependent session type kernel invokes the ECC kernel when it checks an inference rule that contains a functional proof object. Assuming the correctness of the implementation of the kernels, acceptance of a proof object corresponds to a valid derivation of the concluding sequent (including the π/λ -calculus term extraction).

The tactic interface is also split between two layers. Many session type rules in STILL require discharging a functional subgoal in the ECC layer as part of applying the session type rule. The $\$R$ rule of [47], for instance, requires a functional derivation of $\Psi \vdash M : \tau$ before the session type tactic can complete. When such a tactic is invoked, STILL suspends it and opens the functional subgoal under the name `<sessionSubgoal>`. `<functionalSubgoal>`. The user develops the functional proof using the same `apply` command and tactical combinators as the session type layer (see Section 5.1). Once the functional proof is complete, the suspended session type tactic resumes and performs its subgoal expansion and resource reservation. In this way, the two tactic engines share a uniform interactive surface while operating over distinct kernels.

Both kernels contain a Haskell datatype definition for a proof object whose constructors correspond to attempted inference rule applications for the system, and a validation function that determines whether the proof object should be accepted or not, *i.e.*, each inference rule is applied correctly. This includes well-formedness checking of polymorphic session types and checking corecursive calls match the recorded typing environment after substitution. The validation functions in both kernels rely on algorithms for capture-avoiding substitution, α -renaming, and fresh name generation. Fresh name generation is necessary for the completeness of tactic-level delayed partitioning (Section 4.4). The ECC kernel also

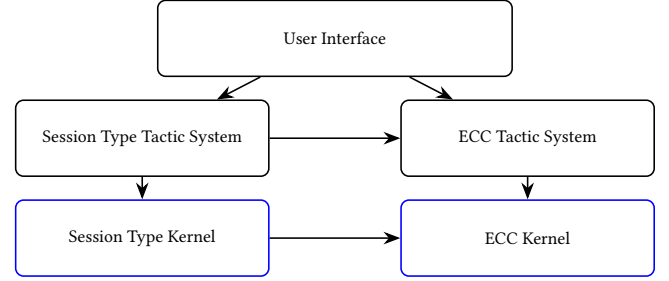


Figure 4. The architecture of the STILL prover.

relies on a normalization algorithm for determining convertibility and universe cumulativity between two terms.

The dependent session type kernel also checks the multiplicative rule applications for disjoint linear contexts between the premises. The resource hiding algorithm prevents the tactic system from constructing such proof objects, but the kernel performs the check independently. While a proof that the hiding algorithm cannot construct an invalid partition is given for the propositional core in Section 4 (Lemma 4.2), the kernel check is nevertheless retained. Tactic implementations are not part of the trusted computing base. Even if a tactic is implemented incorrectly, only kernel validation grants theorem status.

3.4 Design Alternatives

As can be inferred by the brief discussion in Section 3.1, STILL is not the first system to delay context splitting when constructing proofs in linear logic. The I/O model of Hodas and Miller [27] and the resource management systems that refine it [11, 26] also delay context splitting. These systems thread an input context and an output context through the proof rules. The changes these systems make to the proof rules are minor and it is easy to verify they agree with the original presentation of linear logic. STILL distinguishes itself from these systems by providing delayed context splitting in an interactive tactic-based setting. I/O threading is used for backtracking proof search in logic programming interpreters where unification defers the choice of a split and backtracking revisits it. However, in an interactive tactic-based theorem prover, a delayed partition must survive as a stable object in the proof state while the user applies tactics in any order, across any branch, over an arbitrarily long interactive session. The nondeterministic subgoal can be viewed as the interactive counterpart of I/O threading in which each open subgoal carries the currently unclaimed resources and reservations propagate between branches through resource hiding rather than through the input/output contexts of a resource management calculus. Theorem 4.7 in Section 4.5 shows that tactic scripts using STILL's delayed context splitting over the unmodified rules prove exactly the derivable

sequents. Furthermore, the I/O systems verify a second, instrumented presentation against the original calculus while the STILL kernel implements the original calculus directly (see Section 4). Interactive provers for linear logic otherwise either require the user to supply the partition when the tactic is applied [10] or adopt modified rules [50].

Other two-level systems such as Abella [20] and Hybrid [18] place a specification logic inside a reasoning logic and need only one kernel compared to the two-kernel architecture of STILL. A single kernel reduces the amount of trusted code and is a genuine advantage of that architecture. However, the comparison with STILL is not exact. The two levels of Abella and Hybrid are a specification logic and a logic for reasoning about specifications, while the two layers of STILL are cooperating object-level type systems. In STILL, the session layer specifies protocols and the functional layer specifies the values those protocols communicate. STILL is built with a kernel for each level in order to agree with the theory it implements. The dependent session type system of Toninho et al. [47] is parametric in the functional layer, and STILL mirrors this modularity so the ECC layer can be replaced by another dependently typed system without touching the session kernel or tactic engine. The cost of two kernels is a larger trusted code base with two proof object datatypes and two validation functions. The two kernels also implement shared machinery for substitution, α -renaming, and fresh name generation described in Section 3.3. We consider the modularity worth this cost for a system whose purpose is to instantiate a parametric theory.

4 Mechanized Correctness of Delayed Context Partitioning

In this section we describe the verification of the STILL tactic system in Isabelle/HOL [38]. The tactic layer of the STILL prover presents nondeterministic subgoals whose linear contexts overlap across multiplicative branches. The resource reservation system and hiding algorithm work together to ensure that a completed derivation contains disjoint partitions. While the trusted kernel catches tactic implementation mistakes, it does not verify the tactic system works as expected. The mechanization of the STILL tactic system presented in this section establishes that the delayed partitioning mechanism is both sound and complete. Complete tactic derivations are exactly the valid derivations in the logic.

4.1 The Formalized Core

The verification first isolates the fragment of STILL in which the partitioning problem lies. This is propositional intuitionistic linear logic with the multiplicative, additive, and exponential connectives.

$$A, B ::= p \mid 1 \mid A \otimes B \mid A \multimap B \mid A \& B \mid A \oplus B \mid !A$$

Sequents in this fragment have the form $\Delta \vdash A$, where the linear context Δ is a finite map from channel names to types, mirroring the named resources of STILL. The formalization deliberately omits the dependent, polymorphic, and coinductive features, the functional layer, and the π -calculus term assignment. The nondeterministic representation and hiding algorithm concern only the linear context, and none of the omitted features participate in the partitioning argument.

An inductive relation derivable plays the role of the kernel of Section 3.3. The multiplicative rules carry the disjointness side condition explicitly. The rule for a tensor right side derivation is the following:

$$\frac{\Delta_1 \vdash A \quad \Delta_2 \vdash B \quad \text{dom}(\Delta_1) \cap \text{dom}(\Delta_2) = \emptyset}{\Delta_1, \Delta_2 \vdash A \otimes B} \otimes R$$

In this way, the derivable relation performs the same verification that the STILL kernel does against proof objects.

4.2 Completed Subgoal Trees and Resource Hiding

The first layer of the verification is performed against completed subgoal trees (mirroring the subgoal tree of Section 3.3). A function $\text{res}(t)$ computes the resources reserved in a tree, and a predicate $\text{wf}(\Delta, t)$ implements the hiding algorithm of Figure 3. A subgoal may only close on an available resource, and a multiplicative node has each child checked with its sibling's reservations hidden. For example, the clause for a tensor node checks the left child under $\Delta|_{\overline{\text{res}(t_2)}}$ and the right child under $\Delta|_{\overline{\text{res}(t_1)}}$, where $\Delta|_S$ restricts the map Δ to the names in S and $\bar{S}$ is the complement of S . No disjointness condition is stated anywhere in the definition.

The central theorem `tactic_tree_sound` shows that hiding alone is all that is needed to reconstruct a kernel derivation.

Theorem 4.1 (Soundness). *If $\text{wf}(\Delta, t)$ then $\Delta|_{\text{res}(t)} \vdash \text{goal}(t)$ is derivable.*

The proof is done by induction on t , and the multiplicative cases use a lemma, `multiplicative_disjoint`, that formalizes the claim in Section 3.2 that hiding prevents the construction of invalid partitions:

Lemma 4.2 (Derived disjointness). *If $\text{wf}(\Delta, t)$ and t is a multiplicative node with children t_1 and t_2 , then $\text{res}(t_1) \cap \text{res}(t_2) = \emptyset$.*

The reserved sets of multiplicative siblings come out disjoint automatically. A final theorem for the subgoal tree model shows the derivable formulas are exactly the same as the derivable formulas of the formalized core:

Theorem 4.3 (Exact characterization). *$\Delta \vdash A$ is derivable if and only if there is a completed subgoal tree t with $\text{goal}(t) = A$, $\text{res}(t) = \text{dom}(\Delta)$, and $\text{wf}(\Delta, t)$.*

Together, these theorems state that the completed tactic derivations accepted by the hiding mechanism are exactly the derivations that consume the whole linear context.

4.3 The Operational Model

The completed subgoal tree model proves the correctness of a finished tree. We now present a second model for proving the correctness of interactive construction. A datatype for partial derivations extends the subgoal trees of Section 4.2 with open subgoals, and a step relation $p \xrightarrow{\Delta} p'$ between two partial subgoal trees under the initial assumptions in Δ models the application of a single tactic. A single step in the relation expands one open subgoal, and the available resources presented to each subgoal are computed as part of the step relation. This mirrors the way STILL recomputes the available resources after each tactic application.

Three results connect this operational model to the kernel. First, a lemma, `reachable_mult_disjoint`, shows that every partial derivation reachable from an initial goal has disjoint resource reservations across every multiplicative branch. This is the bookkeeping that STILL provides to the user in the middle of the construction of a proof. Second, a theorem, `operational_sound`, shows that completed proofs in the operational model are sound:

Theorem 4.4 (Operational soundness). *If a partial derivation p is complete and accepted by the kernel under Δ , then $\Delta|_{\text{res}(p)} \vdash \text{goal}(p)$ is derivable.*

Third, a theorem, `derivable_reachable_uncond`, shows that the interactive prover can build a proof of anything derivable:

Theorem 4.5 (Operational completeness). *If $\Delta \vdash A$ is derivable, then there is a sequence of tactic steps from the open goal A under Δ to a complete derivation accepted by the kernel.*

Theorems 4.4 and 4.5 show that the proofs that can be constructed by the STILL tactic system are exactly the derivable sequents of the logic.

4.4 The Fresh Name Subtlety

Theorem 4.5 is stated up to the choice of bound channel names. This is essential rather than an artifact of the proof. When a multiplicative expansion is open, the branch completed first is constructed while its sibling has reserved nothing, so its sequents display the full undivided context. Any channel name bound in that first branch must therefore be fresh for the whole context, not merely for the piece the branch will eventually own. A completed subgoal tree that reuses names across a multiplicative split can be a valid proof by Theorem 4.3 but cannot be constructed by any sequence of tactic steps. Trees can be constructed in which each branch binds a name belonging to the other branch's partition, and no interleaving of tactic applications realizes either branch first. The proof of Theorem 4.5 therefore includes an α -renaming development, proving the well-formedness judgment equivariant under channel name permutations and freshening every binder of the witness tree before replaying

it. This finding corroborates the fresh name generation of Section 3.3 as a requirement for the completeness of tactic-level delayed partitioning.

4.5 The Tactic Interface

The step relation of Section 4.3 describes what tactic application does to the proof state. A final layer mechanizes the view of tactics as functions from a goal to a list of subgoals presented by STILL. A nondeterministic goal is a pair (Δ, A) of an availability context and a formula. This is the sequent STILL displays for an open subgoal whose context carries all currently unclaimed resources. Tactics are syntactic rule names carrying the parameters a user supplies (the resource to close on, the channel to decompose, the name of a received channel), and each receives its local meaning from a function `apply_tac` that takes a goal to an optional list of subgoals. In this function applying the $\otimes R$ tactic to $(\Delta, A \otimes B)$ results in $[(\Delta, A), (\Delta, B)]$, both subgoals offered the whole context.

The proof state continues to be the partial derivation from the operational layer. The goal list a user sees is computed from it by the hiding traversal algorithm, and `tac_at` applies a tactic at the i -th open goal. As a result, this layer agrees with the operational step relation exactly. Every successful tactic application is a step, and every step is a tactic application at some position, so the mid-proof disjointness guarantee and Theorem 4.4 transfer verbatim. A theorem called `tac_at_view` proves the correctness of the goal list view:

Theorem 4.6 (Goal-list view). *If applying tactic t at the i -th open goal succeeds, the resulting goal list arises from the previous one by replacing goal i with exactly the subgoals `apply_tac` computes from it. Every other goal keeps its formula, and its available resources can only shrink.*

The shrinking context is the visible effect of the tactic claiming resources. Finally, the `tactics_characterize` theorem proves that scripts of tactics characterize derivability at the level of the user interface:

Theorem 4.7 (Scripts characterize derivability). *$\Delta \vdash A$ is derivable if and only if some tactic script, run from the single open goal A , produces a derivation with no open subgoals and no unreserved resources that consumes $\text{dom}(\Delta)$.*

The condition that no resource is left unreserved cannot be dropped from Theorem 4.7. If that condition is not included, a tactic script can be constructed for the goal $p \multimap 1$ that constructs a derivation with no open subgoals and consumes the (empty) initial context in full, yet the sequent is not derivable. The attempted derivation silently discards the linear resource bound by the $\multimap R$ step. Linearity must be audited on the finished object rather than enforced during construction, because a partial derivation legitimately contains resources it has not reserved yet. This mirrors the

kernel design of Section 3.3, and it shows that even the necessity of the final audit is established by proof rather than left to informal argument.

The layer closes under the tactical combinators provided by STILL. Sequencing, alternation, skip, and repetition tacticals are formalized as transformers of the proof state. Since repetition of a tactical that always succeeds diverges, evaluation cannot be a total function. The mechanization instead gives a big-step evaluation relation in which diverging repetition has no result. Evaluation is made deterministic by having alternation commit to the first branch that succeeds. Additionally, repetition satisfies the defining equation $\text{REPEAT } t = (t \text{ THEN REPEAT } t) \text{ ALT SKIP}$. Correctness is inherited rather than re-proved by showing that every successful tactical run is a sequence of steps of the operational relation. A final theorem called `tactical_characterize` proves that tactical expressions prove exactly the derivable sequents.

4.6 Agreement with a Standard Formalization of ILL

One may ask whether the kernel presented in Section 4.1 is genuinely a presentation of intuitionistic linear logic. In order to answer this, the mechanization presents a two-way bridge to the deep embedding of full ILL in Isabelle/HOL by Smola and Fleuriot [45]. In one direction, a theorem, `derivable_ill`, shows that every kernel derivation can be translated into a derivation of the corresponding sequent in the deep embedding. In the converse direction, a theorem, `fcseq_derivable`, shows that every derivation in the deep embedding that stays within the shared fragment can be replayed as a kernel derivation over any named context that realizes its antecedent multiset. The two calculi can therefore prove exactly the same fragment sequents.

The converse direction relies on a cut-admissibility theorem for the kernel, called `cut_admissible`. The theorem proves the seventeen rules of the kernel admit cut without any extension. The mechanization is performed on a multiset representation of sequents rather than working through the additional complications that would come from performing the mechanization on the named resource presentation. The multiset representation is then translated back to the kernel through name-realization lemmas. Fresh names are re-chosen whenever a concrete context is realized. Only cuts whose cut formula leaves the fragment (through $\top$ and 0) fall outside the correspondence. Closing that gap in the mechanization is left open as the type system STILL implements does not contain $\top$ or 0 .

We emphasize that this mechanization verifies the algorithms of Section 3 and not the Haskell source of STILL itself. Within that scope, this mechanization also demonstrates what the tactic-level placement of the delayed context partitioning achieves over the alternative of modifying the rules themselves, as in leftover typing [1, 53] or the resource management systems of Section 3.4. For a finished derivation, a modified calculus records nearly the same information as our

reservations, and such a presentation of delayed partitioning can be made to work interactively [50]. The results presented in this section separate the two designs. Disjointness of multiplicative partitions is a theorem about well-formed trees (Lemma 4.2) rather than a side condition carried by every multiplicative rule. The global fresh name requirement in Section 4.4 is a property of branch construction order that only arises because the proof state exists over time. A derivation in a modified calculus is a static object with no analogous notion. Finally, since the rules are untouched, agreement with the standard sequent calculus is a theorem about the original system itself, cut admissibility included, rather than a correspondence between the original system and another modified one.

5 Evaluating the System

In this section we describe how users interact with STILL, and present our evaluation of the faithfulness and usefulness of the theorem prover. STILL runs in four modes: the script mode for batch proof checking and benchmarking, the file watch mode that reruns the proof script as the file is updated, the REPL mode for experimentation, and a simple server mode that allows IDE users and large language models [36] to inspect the proof state after each command in the proof script. The resource hiding algorithm operates in all execution modes.

5.1 Working With STILL

Figure 5 shows a REPL session at an intermediate state of the proof, alongside the full proof script. Together these demonstrate the usefulness of resource hiding. The REPL display prints the contexts $\Omega; \Psi; \Gamma; \Delta$ (empty contexts are left blank); open subgoals are prefixed by $?$, and the current subgoal is marked with $*$. The same proof state display is available to external editors via the STILL LSP-style server mode. The REPL shows the result of applying the `TensorR` tactic to the goal $A; \cdot; \cdot; a:A, b:A \vdash z : A \otimes A$. After the tactic is applied, two nondeterministic subgoals are created, $?d$ and $?e$, each displaying the full linear context with the channels that have not been claimed in red. No partition has been determined. Applying `IdA3` closes $?e$ by consuming one of the two available resources of type A . The system records that reservation, and now when the user views the $?d$ subgoal only the unreserved channel b remains and is colored black to indicate there are no open multiplicative branches that also have b available in the linear context.

This minimal example already demonstrates the usefulness of the STILL tactic engine. No partition of the linear

³The automatic version of the `Id` tactic. Automatic versions of tactics that use resources in the contexts are provided in addition to versions that allow specifying which resource to use.

```

*?c>> A; ; ; a:A, b:A |- z:A ⊗ A
π> apply TensorR
Tensor right side tactic applied.
*?e>> A; ; ; a:A, b:A |- z:A
?d>> A; ; ; a:A, b:A |- c:A
π> apply IdA
IdA tactic applied.
*?d>> A; ; ; b:A |- c:A
π>

```

```
assume A is stype
```

```

theorem bookkeeping: "A * A -o A * A"
apply ImpliesR
apply TensorLA
apply TensorR
apply IdA
apply IdA
done

```

Figure 5. A STILL REPL session and proof script illustrating delayed partitioning.

Module	Theorems	Total Subgoals Total Proof Rules	Max Subgoals Max Proof Rules	Time (s)
Auction	3	104 117	38 49	0.004088
Bank	2	51 102	30 66	0.002425
CloudServer	2	25 25	14 14	0.000393
Counter	3	72 136	32 89	0.001473
LargeMul	1	3997 2999	3997 2999	1.929270

Table 1. Benchmarks for the STILL prover.

context had to be manually supplied, and tactics that consume resources did not have to be supplied the correct unclaimed resource. Without the resource hiding algorithm, the user would have to track which occurrence of A had been consumed in the sibling branch, or provide the partition when the `TensorR` tactic is applied. In larger proofs with tactic combinators that expand many subgoals at once, this bookkeeping is eliminated by the STILL system.

5.2 Example Proofs and Benchmarks

The STILL source code comes with example proofs that demonstrate its features⁴. The proofs in Table 1 evaluate the faithfulness of the implementation of the underlying logic in the STILL kernel along with a demonstration of how the resource hiding algorithm scales beyond real protocols.

The Bank, CloudServer, and Counter modules assess the faithfulness of the implementation of the theory of dependent session types with polymorphism and coinduction. The Bank module validates a bank and ATM client protocol from [47]. The CloudServer module validates a polymorphic cloud server protocol, and a file conversion instance of the protocol from [7]. The Counter module validates a coinductive protocol from [48] for a bit counter which includes processes that store bits and are spawned as more storage is required.

The LargeMul module stress tests the resource hiding traversal algorithm in the worst-case scenario: traversing the entire subgoal tree for every tactic application. It contains an expression with 1000 variables combined using the tensor

operator. Implication of the same expression, but reversed, allows the proposition to be proved, *i.e.*, $(a * \dots * 138)$ implies $(138 * \dots * a)$. This module demonstrates that the traversal algorithm for reserving and hiding resources does not impede proof checking and development in a meaningful way. We are not aware of a real-world protocol that splits the linear context into one thousand partitions. The proof of the proposition also demonstrates the power of the proposition also demonstrates the power of the nondeterministic subgoals. The tactic expression `ImpliesR; TensorLA+; (TensorR; (IdA <|> Skip))+` proves the entire proposition.

Additional experiments testing how performance scales can be found in the STILL source code. Tensor expressions like those in the LargeMul module are evaluated with sizes from 10 to 1000, and the results show quadratic scaling. These are worst-case benchmarks as the tensor operator expands subgoals with multiplicative branches in the subgoal tree, so every subgoal must be traversed to hide resources for each tactic application. Further work is required to see if performance can be improved.

5.3 Agent-Driven Proof Development

In addition to the human-driven benchmarks above, we evaluate the STILL tactic interface by treating it as a target for an LLM coding agent. This evaluation was performed to test that the tactic engine is usable enough to be driven by a user with no prior exposure to STILL. Claude Opus 4.7 [2], accessed in May 2026, was provided a harness to check and inspect STILL proof scripts, a tutorial document, the tactics documentation, and a reference to the concrete syntax of proof scripts. Two sessions were performed that we report as qualitative observations.

In the first session, the agent attempted the Bank [47], CloudServer [7], and bit counter [48] protocols. All three were eventually accepted by the kernel, but the bit counter was not built to spawn new nodes on increment. The agent instead built a counter with a fixed size of three bits. After we expanded the tutorial with additional material on coinductive proof patterns, a second attempt produced the dynamic node-spawning counter. This suggests the tutorial, not the tactic interface, was the limiting factor.

A second coding session was performed by having the agent specify and verify a TCP-like handshake protocol. The

⁴The proof scripts can be executed by running `./still Proofs/*.still` and a benchmark table can be generated by running `./still benchmark Proofs/*.still`.

agent was provided the same tooling with the updated tutorial. At first, a correct but weaker specification without dependent types was verified by the agent. When prompted to use dependent types to enforce correct sequence numbering, the agent revised the protocol and produced a proof in which the type of the ACK channel depends on the SYN value. This demonstrates that the agent could exercise the dependency features once prompted toward them, but may not always design a protocol with dependency in mind.

That the agent completed the proofs successfully indicates the tactic interface is comprehensible enough to be driven by a non-human reasoner without prior exposure to STILL beyond the documentation. The interventions for the dynamic counter and the dependent handshake stemmed from specification issues, not tactical ones. The agent could construct proofs once pointed at the right specification. The kernel rejected 14 intermediate proof scripts across the two sessions. Every finished proof was accepted. These brief experiments reflect the capabilities of one model at one point in time. We believe that an interface operable by a present-day agent from its documentation will remain so as models improve. The evaluation does not show that agents can verify STILL protocols that are substantially harder than the ones in the source papers. That is a question left for future work.

5.4 The Auction Case Study

Figure 6 presents the English auction protocol from the Auctions module (omitting the proof for space), and demonstrates the power of dependent session types alongside coinductive and polymorphic session types. The auction protocol begins with the seller providing the type of the item being auctioned. The item type has type `stype`, so it represents a service being offered (including ones that may transfer a tangible object). Then the protocol introduces a type for bids, a bid validation type family, and a currency type family. The bid validation type family represents a validation parameterized by the old bid and the current bid. A term of the type `validNewB b1 b2` is a proof that `b2` is a valid new bid when checked against the previous bid `b1`. The currency dependent type family represents the currency being provided, and is parameterized by the amount of the currency. With this, the auction is polymorphic with respect to the item being offered, the validation of the bids, and the type of the currency. Finally, before bidding begins, the item is sent to the auction process, and the seller provides a channel that will receive the winning bid.

After the auction begins, the first bid amount can be provided along with a validation of the first bid against the zero bid. After that, the correct amount of currency must be provided. The current highest bidder also provides a channel that can either receive the item in the case that they win, or their money back in the case that their bid is beaten. After

```
theorem english:
  "forall ItemType:stype. forall bid:Type 1. forall zeroB:bid.
   forall validNewB:(Pi b1:bid. Pi b2:bid. Type 1).
   forall currency:(Pi b:bid.Type 1).ItemType -o
   (forall b:bid. $currency b -o 1) -o forall firstB:bid.
   forall firstValid:validNewB zeroB firstB.
   $currency firstB -o
   (ItemType + $currency firstB -o 1) -o
   nu Y. (forall newB:bid.
   forall nextValid:validNewB firstB newB.
   $currency newB -o
   (ItemType + $currency newB -o 1) -o Y) & 1"

z(ItemType).z(bid).z(zeroB).z(validNewB).z(currency).z(a).z(c).
z(firstB).z(firstValid).z(d).z(e).
(corec AUC(item, ownerC, curB, winnerC, oldB, lastB, curValid).
  z.case(
    inl: z(newB).z(nextValid).z(h).z(i).((v j) winnerC[j].
      (j.inr;[curB <-> j]
        | AUC(item, ownerC, h, i, lastB, newB, nextValid))),
    inr: ownerC[lastB].((v f) ownerC[f].([curB <-> f]
      | (v g) winnerC[g].(g.inl;[item <-> g] | 0))))))
(a, c, d, e, zeroB, firstB, firstValid)
```

Figure 6. English auction specification and extracted π -calculus term.

the first bid is fully processed, the protocol enters a coinductive session that repeats the process of receiving a bid, or terminates once a termination signal is given.

The linearity of the session types ensures that once another bid is received, the money is returned to the previous highest bidder. The dependent session types ensure that bidders only receive their money back, because the refund channel is typed with the current bid amount. No other amount of currency can be sent back to the bidder. Additionally, the seller can only receive the highest bid at the end of the auction, because the coinductive session type requires that channels that exist at the start of the auction remain available as the auction continues. Finally, when the auction receives the signal to terminate (*i.e.*, enters the right side of the $\&$ connective), the item must be sent to the highest bidder, because that is the only channel that remains that can receive the item, and the seller receives the money from the highest bid, because that is the only channel that remains that can receive money.

The π -calculus process extracted from the proof of the validity of the auction protocol can also be seen in Figure 6. It has been formatted for readability and to save space. It enjoys the properties explained above that have been guaranteed by the dependent session types: bids must be validated, losers receive their money back, the winner receives the item, and the seller of the item receives the money from the winning bid.

5.5 Evaluation Summary

The scope of the evaluation presented here is deliberate. Its aim is to show that STILL faithfully checks the motivating protocols of the source papers that describe the underlying logic [7, 47, 48], that one new dependently typed protocol can be specified and verified from scratch, that the resource

hiding mechanism remains usable in a proof with a number of multiplicative branches beyond what real-world protocols contain, and that the tactic interface can be operated without prior exposure to the system. It does not show that STILL supports large multi-module developments, and the protocols verified remain of the size found in the session type literature. A substantial case study beyond that literature is future work.

6 Related Work

Implementations of dependent session types are most closely related to the work presented in this paper. In particular, the work by Ciccone and Padovani [14] presents a dependently typed π -calculus in Agda. Their system also includes a layer for a functional type system that is separate from the session type layer in the same way the system presented by Toninho et al. [47] does. Because the system is embedded in Agda, context partitioning for the typing of parallel composition is witnessed explicitly at the term level (*i.e.*, the partition is included in the proof term arising from the application of the rule). In STILL, partitioning is delayed and resolved in the tactic layer, and disjointness obligations are verified by the kernel once a proof object is constructed. Other implementations of dependent session types can be found in the RAST [17] and Granule [32] programming languages. RAST provides type-level dependency with arithmetic refinements, and Granule provides type-level indexing using natural numbers. There are also many programming languages supporting implementations of session types [19]. To our knowledge, none of these are dependently typed.

Due to the Curry-Howard correspondence identified between linear logic and session types, previous work on interactive theorem proving for linear logic is also closely related to the work presented in this paper. A system for interactive development of intuitionistic linear logic has been implemented in Isabelle by Kalvala and De Paiva [29], and a deep embedding of intuitionistic linear logic in Isabelle has been developed by Smola and Fleuriot [45]. Sano et al. [44] have also presented a mechanization of classical linear logic in the Beluga [43] proof assistant. Standalone provers have also been implemented for linear logic. These systems include the Janet theorem prover developed by Trifunovski [50], and the Click and Collect theorem prover by Callies and Laurent [10]. The Click and Collect prover requires the partition of the linear context to be provided by the user at the time multiplicative tactics are applied. The other previously mentioned systems modified the inference rules in the system to allow for delayed context partitioning.

Actris [25] brings session-type style protocols into the Iris separation logic framework in Rocq. Protocols in Actris are specified as dependent separation protocols, and concurrent programs in a realistic language are verified against them alongside higher-order state, locks, and other concurrency

features. This type of embedding allows Actris to model programming constructs that STILL does not, and it inherits the mature proof automation of its host. In Actris, a program is written and then proved to follow a protocol, while in STILL, the derivation of a session type is itself the process that behaves according to the protocol. Iris also has no counterpart to the delayed context partitioning that STILL provides at the tactic level.

Related to specification of protocols for concurrent processes using the STILL prover is the Concurrent Logical Framework (CLF) developed by Watkins et al. [52], which provides a dependent framework for specifying concurrent systems with linear resources. Examples are given that embed asynchronous and synchronous variants of the π -calculus inside CLF [12]. Cervesato and Sacchini extend this line of work with Meta-CLF [13], which supports meta-reasoning over CLF specifications. Together, CLF provides a framework for specifying concurrent operational semantics and Meta-CLF provides a framework for reasoning about those specifications, whereas STILL provides interactive proof development in an object-level dependent session type theory.

Also tangentially related to the STILL theorem prover is Idris 2 [6], which is built on Quantitative Type Theory [3, 33]. While it is not directly related to session types or linear logic, Idris 2 presents a dependently typed language and proof environment that must track resources as they are consumed. However, Quantitative Type Theory is not a theory of session types (there is no context partitioning for the $\otimes$ rule), and Idris 2 is not a tactic-based theorem prover, so the relation between STILL and Idris 2 is marginal.

Proof search systems for linear logic and type checking implementations of session types are also related to our work on delayed context partitioning. Proof search systems include the work by Hodas and Miller [27] and Cervesato et al. [11] on efficient resource management for the Lolli prover. The work of Harland et al. [23] on the Lygon theorem prover also identifies the problem of finding the correct context partition for the multiplicative rules. Allais [1] presents a type checker for a linear λ -calculus using leftovers. Zalakain and Dardha [53] use the same techniques to implement a type checker for a π -calculus. The resource management calculus provided by Cervesato et al. [11] and the leftover type system presented by Allais [1] both result in systems that return the resources that were not consumed in one branch of the premises of a multiplicative rule. These are systems that behave similarly to the way the STILL prover hides resources; however, the delayed partitioning is encoded in the calculus/type system, while the STILL prover delays the choice at the tactic level and validates the resulting partitions at the kernel level.

7 Conclusions and Future Work

To our knowledge, the STILL prover is the first standalone tactic-based interactive theorem prover for dependent session types. Its central contribution is to represent linear context partitions nondeterministically in the tactic system rather than modifying the proof rules to accommodate delayed partitioning. The tactic engine prunes partition choices as they are ruled out by later tactic applications, and the kernel validates partitions only when a proof is complete. This technique preserves the metatheory of the underlying calculi [7, 47, 48] and should transfer to other tactic-based theorem provers for substructural logics. The mechanization of Section 4 gives the correctness of the technique formal backing for the propositional core, and shows that the STILL tactic system proves exactly the derivable sequents of the logic. Extending the mechanization to the dependent, polymorphic, and coinductive features of STILL is natural future work.

There are several directions to extend the work presented in this paper. The π -calculus terms STILL extracts are not directly executable. Translating them to a target like the Session Abstract Machine [9], ProVerif [5], or a smart contract programming language based on process calculi [4, 16, 34] would allow extracted proofs to deliver the guarantees of their protocols in a running system. Translating the English auction of Section 5.4 into a smart contract would provide the auction's liquidity, refund, and asset transfer properties on-chain. Formal verification of the translation would be required to preserve the type-level invariants.

The evaluation of agent-driven proof development of Section 5.3 is a starting point. Open questions include how agent performance scales with protocol complexity and whether the resource hiding is what makes the tactic interface usable for a non-human reasoner. Disabling resource hiding and comparing the performance of the agent on the same proofs would test the usefulness of resource hiding for agents directly. It is interesting to note that if the prompt does not mention Curry-Howard, Claude [2] does not develop proofs with Curry-Howard in mind, but rather keeps a sharp distinction between terms and proofs.

Future work also remains in the functional layer of STILL. The modular architecture of STILL is intended to admit the replacement of the ECC functional layer with another dependently typed system, e.g., Rocq [46], LF [24], Twelf [42], or Agda [40]. This would require adapting the tactic suspension mechanism described in Section 3.3 to the architecture of the replacement system. We expect this to take significant effort.

In terms of broader applications of the work presented here, classical session type systems [51] and other substructural type theories are plagued with the same partitioning issue that STILL addresses. The technique of using nondeterministic subgoals is not limited to the intuitionistic fragment of linear logic. We expect the same tactic implementation

techniques can be used to implement other flavors of substructural logic in their own tactic-based theorem provers without changing the underlying proof rules.

Use of AI tools. The code and mechanization for the STILL theorem prover were developed with the help of generative AI tools. For the STILL theorem prover, the test suite, script interpreter, file watch, REPL, and MCP integration were created with the help of generative AI (Claude, ChatGPT, Gemini). For the Isabelle/HOL verification, the authors specified the definitions, the theorem statements, and the proof architecture, in addition to directing the development and auditing the resulting theories. Coding agents (Claude) produced much of the Isabelle proof text under the direction of the authors. Every proof is checked by the Isabelle kernel, and the development contains no unproven assumptions. AI tools were also used to draft prose that the authors rewrote (Claude, ChatGPT, Gemini). The authors take full responsibility for all content of this paper.

References

- [1] Guillaume Allais. 2019. Typing with Leftovers – A Mechanization of Intuitionistic Multiplicative-Additive Linear Logic. In *23rd International Conference on Types for Proofs and Programs (TYPES 2017) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 104)*, Andreas Abel, Fredrik Nordvall Forsberg, and Ambrus Kaposi (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 1:1–1:22. doi:10.4230/LIPIcs.TYPES.2017.1
- [2] Anthropic. 2026. System Card: Claude Opus 4.7. <https://cdn.sanity.io/files/4zrzovbb/website/037f06850df7f8e871e206dad004c3db5fd50340.pdf>
- [3] Robert Atkey. 2018. Syntax and Semantics of Quantitative Type Theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS '18)*. Association for Computing Machinery, New York, NY, USA, 56–65. doi:10.1145/3209108.3209189
- [4] Massimo Bartoletti, Riccardo Marchesin, and Roberto Zunino. 2024. Secure Compilation of Rich Smart Contracts on Poor UTXO Blockchains. In *2024 IEEE 9th European Symposium on Security and Privacy (EuroS&P)*. IEEE, Piscataway, NJ, USA, 235–267. doi:10.1109/EuroSP60621.2024.00021
- [5] Bruno Blanchet, Martin Abadi, and Cédric Fournet. 2008. Automated verification of selected equivalences for security protocols. *The Journal of Logic and Algebraic Programming* 75, 1 (Feb. 2008), 3–51. doi:10.1016/j.jlap.2007.06.002
- [6] Edwin Brady. 2021. Idris 2: Quantitative Type Theory in Practice. In *35th European Conference on Object-Oriented Programming (ECOOP 2021) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 194)*, Anders Möller and Manu Sridharan (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 9:1–9:26. doi:10.4230/LIPIcs.ECOOP.2021.9
- [7] Luís Caires, Jorge A. Pérez, Frank Pfenning, and Bernardo Toninho. 2013. Behavioral Polymorphism and Parametricity in Session-Based Communication. In *Programming Languages and Systems*, Matthias Felleisen and Philippa Gardner (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 330–349.
- [8] Luís Caires and Frank Pfenning. 2010. Session Types as Intuitionistic Linear Propositions. In *CONCUR 2010 - Concurrency Theory*, Paul Gastin and François Laroussinie (Eds.). Springer, Berlin, Heidelberg, 222–236. doi:10.1007/978-3-642-15375-4_16
- [9] Luís Caires and Bernardo Toninho. 2024. The Session Abstract Machine. In *Programming Languages and Systems*, Stephanie Weirich (Ed.).

- Springer Nature Switzerland, Cham, 206–235.
- [10] Etienne Callies and Olivier Laurent. 2021. Click and coLLecT An Interactive Linear Logic Prover. In *5th International Workshop on Trends in Linear Logic and Applications (TLLA 2021)*. Rome (virtual), Italy. <https://hal-lirmm.ccsd.cnrs.fr/lirmm-03271501>
 - [11] Iliano Cervesato, Joshua S. Hodas, and Frank Pfenning. 1996. Efficient Resource Management for Linear Logic Proof Search. In *Extensions of Logic Programming*, Roy Dyckhoff, Heinrich Herre, and Peter Schroeder-Heister (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 67–81. doi:10.1007/3-540-60983-0_5
 - [12] Iliano Cervesato, Frank Pfenning, David Walker, and Kevin Watkins. 2002. *A Concurrent Logical Framework II: Examples and Applications*. Technical Report CMU-CS-02-102. Department of Computer Science, Carnegie Mellon University.
 - [13] Iliano Cervesato and Jorge Luis Sacchini. 2013. Towards Meta-Reasoning in the Concurrent Logical Framework CLF. In *Proceedings Combined 20th International Workshop on Expressiveness in Concurrency and 10th Workshop on Structural Operational Semantics*, Buenos Aires, Argentina, 26th August, 2013 (*Electronic Proceedings in Theoretical Computer Science*, Vol. 120), Johannes Borgström and Bas Luttik (Eds.). Open Publishing Association, 2–16. doi:10.4204/EPTCS.120.2
 - [14] Luca Ciccone and Luca Padovani. 2020. A Dependently Typed Linear π -Calculus in Agda. In *Proceedings of the 22nd International Symposium on Principles and Practice of Declarative Programming* (Bologna, Italy) (PPDP '20). Association for Computing Machinery, New York, NY, USA, Article 8, 14 pages. doi:10.1145/3414080.3414109
 - [15] R. L. Constable, S. F. Allen, H. M. Bromley, W. R. Cleaveland, J. F. Cremer, R. W. Harper, D. J. Howe, T. B. Knoblock, N. P. Mendler, P. Panangaden, J. T. Sasaki, and S. F. Smith. 1986. *Implementing Mathematics with the Nuprl Proof Development System*. Prentice-Hall, Inc., USA.
 - [16] Ankush Das, Stephanie Balzer, Jan Hoffmann, Frank Pfenning, and Ishani Santurkar. 2021. Resource-Aware Session Types for Digital Contracts. In *2021 IEEE 34th Computer Security Foundations Symposium (CSF)*. IEEE, Piscataway, NJ, USA, 1–16. doi:10.1109/CSF51468.2021.00004
 - [17] Ankush Das and Frank Pfenning. 2022. Rast: A Language for Resource-Aware Session Types. *Logical Methods in Computer Science* 18, 1 (Jan. 2022). doi:10.46298/lmcs-18(1:9)2022
 - [18] Amy Felty and Alberto Momigliano. 2012. Hybrid: A Definitional Two-Level Approach to Reasoning with Higher-Order Abstract Syntax. *Journal of Automated Reasoning* 48, 1 (Jan. 2012), 43–105. doi:10.1007/s10817-010-9194-x
 - [19] Simon Fowler. 2020. From Data Types to Session Types: A Basis for Concurrency and Distribution (ABCD). <https://groups.inf.ed.ac.uk/abcd/index.html> Project website.
 - [20] Andrew Gacek. 2008. The Abella Interactive Theorem Prover (System Description). In *Automated Reasoning*, Alessandro Armando, Peter Baumgartner, and Gilles Dowek (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 154–161.
 - [21] Jean-Yves Girard. 1987. Linear logic. *Theoretical Computer Science* 50, 1 (Jan. 1987), 1–101. doi:10.1016/0304-3975(87)90045-4
 - [22] Michael J. Gordon, Arthur J. Milner, and Christopher P. Wadsworth. 1979. *Edinburgh LCF*. Lecture Notes in Computer Science, Vol. 78. Springer, Berlin, Heidelberg. doi:10.1007/3-540-09724-4
 - [23] James Harland, David Pym, and Michael Winikoff. 1996. Programming in Lygon: An Overview. In *Algebraic Methodology and Software Technology*, Martin Wirsing and Maurice Nivat (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 391–405. doi:10.1007/BFb0014329
 - [24] Robert Harper, Furio Honsell, and Gordon Plotkin. 1987. A Framework for Defining Logics. In *Proceedings of the Second Annual IEEE Symposium on Logic in Computer Science (LICS 1987)*. IEEE, Piscataway, NJ, USA, 194–204.
 - [25] Jonas Kastberg Hinrichsen, Jesper Bengtson, and Robbert Krebbers. 2019. Actris: session-type based reasoning in separation logic. *Proc. ACM Program. Lang.* 4, POPL, Article 6 (Dec. 2019), 30 pages. doi:10.1145/3371074
 - [26] Joshua S. Hodas, Pablo López, Jeffrey Polakow, Lubomira Stoilova, and Ernesto Pimentel. 2002. A Tag-Frame System of Resource Management for Proof Search in Linear-Logic Programming. In *Computer Science Logic*, Julian Bradfield (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 167–182. doi:10.1007/3-540-45793-3_12
 - [27] J. S. Hodas and D. Miller. 1994. Logic Programming in a Fragment of Intuitionistic Linear Logic. *Information and Computation* 110, 2 (May 1994), 327–365. doi:10.1006/inco.1994.1036
 - [28] Kohei Honda. 1993. Types for dyadic interaction. In *CONCUR'93*, Eike Best (Ed.). Springer, Berlin, Heidelberg, 509–523. doi:10.1007/3-540-57208-2_35
 - [29] Sara Kalvala and Valeria De Paiva. 1995. Mechanizing Linear Logic in Isabelle. In *10th International Congress of Logic, Methodology and Philosophy of Science*, Vol. 24.
 - [30] Zhaohui Luo. 1990. *Extended calculus of constructions*. Ph.D. Dissertation. The University of Edinburgh. <https://era.ed.ac.uk/handle/1842/12487>
 - [31] Zhaohui Luo and Robert Pollack. 1992. *LEGO Proof Development System: User's Manual*. LFCS, Department of Computer Science, University of Edinburgh, Edinburgh, UK.
 - [32] Danielle Marshall and Dominic Orchard. 2024. Non-linear communication via graded modal session types. *Information and Computation* 301 (2024), 105234. doi:10.1016/j.ic.2024.105234
 - [33] Conor McBride. 2016. *I Got Plenty o' Nuttin'*. Springer International Publishing, Cham, 207–233. doi:10.1007/978-3-319-30936-1_12
 - [34] Kegan McIlwaine and James Caldwell. 2025. Verifying Smart Contract Transformations Using Bisimulations. In *6th International Workshop on Formal Methods for Blockchains (FMBC 2025) (Open Access Series in Informatics (OASIs), Vol. 129)*, Diego Marmosier and Meng Xu (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 9:1–9:19. doi:10.4230/OASIs.FMBC.2025.9
 - [35] Robin Milner. 1999. *Communicating and mobile systems: the π -calculus*. Cambridge University Press, USA.
 - [36] Model Context Protocol. 2025. Model Context Protocol Specification, Revision 2025-11-25. <https://modelcontextprotocol.io/specification/2025-11-25>
 - [37] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction – CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings*. Springer-Verlag, Berlin, Heidelberg, 625–635. doi:10.1007/978-3-030-79876-5_37
 - [38] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. 2002. *Isabelle/HOL - A Proof Assistant for Higher-Order Logic*. Lecture Notes in Computer Science, Vol. 2283. Springer, Berlin, Heidelberg. doi:10.1007/3-540-45949-9
 - [39] Bengt Nordström, Kent Petersson, and Jan M Smith. 1990. *Programming in Martin-Löf's type theory*. Vol. 200. Oxford University Press, Oxford.
 - [40] Ulf Norell. 2007. *Towards a practical programming language based on dependent type theory*. Ph.D. Dissertation. Chalmers University of Technology, Göteborg, Sweden. <https://www.cse.chalmers.se/~ulfn/papers/thesis.pdf>
 - [41] Lawrence C. Paulson. 1993. Isabelle: The Next 700 Theorem Provers. *CoRR* cs.LO/9301106 (1993). doi:10.48550/arXiv.cs/9301106
 - [42] Frank Pfenning and Carsten Schürmann. 1999. System Description: Twelf – A Meta-Logical Framework for Deductive Systems. In *Automated Deduction – CADE-16*. Springer Berlin Heidelberg, Berlin, Heidelberg, 202–206.
 - [43] Brigitte Pientka. 2008. A type-theoretic foundation for programming with higher-order abstract syntax and first-class substitutions. In *Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on*

- Principles of Programming Languages* (San Francisco, California, USA) (POPL '08). Association for Computing Machinery, New York, NY, USA, 371–382. doi:10.1145/1328438.1328483
- [44] Chuta Sano, Ryan Kavanagh, and Brigitte Pientka. 2023. Mechanizing Session-Types using a Structural View: Enforcing Linearity without Linearity. *Proc. ACM Program. Lang.* 7, OOPSLA2, Article 235 (Oct. 2023), 26 pages. doi:10.1145/3622810
- [45] Filip Smola and Jacques D. Fleuriot. 2024. Linear Resources in Isabelle/HOL. *J. Autom. Reason.* 68, 2 (May 2024), 49 pages. doi:10.1007/s10817-024-09698-2
- [46] The Rocq Development Team 2025. *The Rocq Prover*. The Rocq Development Team. doi:10.5281/zenodo.17473943
- [47] Bernardo Toninho, Luís Caires, and Frank Pfenning. 2011. Dependent session types via intuitionistic linear type theory. In *Proceedings of the 13th international ACM SIGPLAN symposium on Principles and practices of declarative programming*. ACM, Odense Denmark, 161–172. doi:10.1145/2003476.2003499
- [48] Bernardo Toninho, Luis Caires, and Frank Pfenning. 2014. Corecursion and Non-divergence in Session-Typed Processes. In *Trustworthy Global Computing*, Matteo Maffei and Emilio Tuosto (Eds.). Springer, Berlin, Heidelberg, 159–175. doi:10.1007/978-3-662-45917-1_11
- [49] Bernardo Toninho, Luís Caires, and Frank Pfenning. 2021. A Decade of Dependent Session Types. In *Proceedings of the 23rd International Symposium on Principles and Practice of Declarative Programming (PPDP '21)*. Association for Computing Machinery, New York, NY, USA, 1–3. doi:10.1145/3479394.3479398
- [50] Maksim Trifunovski. 2017. *An Interactive Proof Assistant for Linear Logic*. Bachelor of Arts with Departmental Honors. Wesleyan University, Middletown, CT. doi:10.14418/wes01.1.1339
- [51] Philip Wadler. 2012. Propositions as sessions. In *Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming (Copenhagen, Denmark) (ICFP '12)*. Association for Computing Machinery, New York, NY, USA, 273–286. doi:10.1145/2364527.2364568
- [52] Kevin Watkins, Iliano Cervesato, Frank Pfenning, and David Walker. 2003. *A Concurrent Logical Framework I: Judgments and Properties*. Technical Report CMU-CS-02-101. Department of Computer Science, Carnegie Mellon University.
- [53] Uma Zalakain and Ornela Dardha. 2021. π with Leftovers: A Mechanisation in Agda. In *Formal Techniques for Distributed Objects, Components, and Systems: 41st IFIP WG 6.1 International Conference, FORTE 2021, Held as Part of the 16th International Federated Conference on Distributed Computing Techniques, DisCoTec 2021, Valletta, Malta, June 14–18, 2021, Proceedings*. Springer-Verlag, Berlin, Heidelberg, 157–174. doi:10.1007/978-3-030-78089-0_9